

Kolokwium z programowania współbieżnego

Zadania główne i bonusowe

Zadanie 1

Co to są *własne sprawy* i *sekcja krytyczna*? (1 pkt)

Jakie są typowe założenia dotyczące zachowania *własnych spraw*? (1 pkt)

Czy podany program jest bezpieczny i/lub żywotny? Jeśli nie, podaj kontrprzykład (przeplot). Jeśli tak, podaj formalny dowód. (8 pkt)

```
1 #include <iostream>
2 #include <thread>
3 #include <atomic>
4
5 std::atomic<int> who_waits{1};
6
7 void process(int id) {
8     while (true) {
9         // własne sprawy
10        while (who_waits == id) {} //protokół wstępny
11        // sekcja krytyczna
12        who_waits = id; // protokół końcowy
13    }
14 }
15
16 int main() {
17     std::jthread t1(process, 0);
18     std::jthread t2(process, 1);
19     return 0;
20 }
```

Uwaga. Przy podawaniu definicji proszę być precyzyjnym. Na przykład pojęcia takie jak *jednoczesny dostęp* itp. wymagają formalnego zdefiniowania przed ich użyciem, jeśli są stosowane w kontekście wątków lub procesów.

Zadanie 2

Problem uczujących filozofów (Dining philosophers) dla $n \in \mathbb{N}$ filozofów. Co to jest graf przydziału zasobów (Resource Allocation Graph, RAG)? Jak w takim grafie definiuje się deadlock? Co to jest graf wait-for? Mając graf wait-for, jak definiuje się cykliczne oczekiwanie? (2 pkt)

W terminach RAG sformułuj i udowodnij twierdzenie Coffmana o warunkach koniecznych deadlocka. (6 pkt) Jak można wykorzystać to twierdzenie do rozwiązania problemu dining philosophers? Podaj implementację takiego rozwiązania w C++. (2 pkt)

Wersja awaryjna na 4pkt (bez grafu RAG) zamiast 10pkt. Podaj warunki Coffmana i je udowodnij dla 5 uczujących filozofów. Jak można wykorzystać to twierdzenie do rozwiązania problemu dining philosophers? Podaj implementację takiego rozwiązania w C++.

Zadania bonusowe

Za każde poprawnie rozwiązane zadanie można zdobyć dodatkowe 1 lub 2 pkt.

- a) Co to jest *livelock*? Podaj przykład współbieżnego programu w C++, który obrazuje *livelock*. Wykaż, że *livelock* rzeczywiście zachodzi w Twoim przykładzie. (1pkt)
- b) Co to jest wzorec *compare-and-swap*? Jak użyć go do zaimplementowania w C++ funkcji `max` działającej na wielu wątkach? Wyjaśnij krok po kroku kluczowe przepłyty, które mogą wystąpić w tej implementacji. (1pkt)
- c) Co to jest prawo Amdahla? Wyprowadź ten wzór krok po kroku i wyjaśnij intuicyjnie znaczenie występujących w nim wyrażeń. Podaj przykłady zastosowania tego wzoru: kiedy warto, a kiedy nie warto zrównoleglić program. (1pkt)
- d) Co to jest *thread pool*? Jaka jest motywacja używania tej struktury? Co to są `std::future` i `std::promise`? Zaimplementuj za ich pomocą w C++ strukturę `ThreadPool`. (2pkt)
- e) Co to jest *ring buffer* (bufor kołowy)? Kiedy i w jakim celu się go używa? Zaimplementuj w C++ thread-safe strukturę `RingBuffer` z użyciem `std::mutex` oraz `std::condition_variable` (konstruktor oraz metody `push` i `pop`). Podaj przykład programu C++ używającego `RingBuffer` (ilustrującego jego użyteczność). (2pkt)